TP Image : apprentissage de représentations par des approches auto-supervisées

1 Objectif du TP

Le TP repose sur les démos du cours CS294-158 de Berkeley. On y trouve trois tâches prétextes déjà pré-entraînées sur CIFAR-10 : le *context encoder* [1], la prédiction de rotation [2] et SimCLR [3]. Nous avons commencé par lancer chaque démo pour voir ce qu'elles produisent. Nous avons ensuite écrit un script qui évalue les trois représentations de la même manière et regroupe les résultats dans un seul tableau, sur CIFAR-10 puis sur STL-10. Enfin, nous avons implémenté une quatrième tâche prétexte, la prédiction de la position relative d'un patch [4], que nous avons entraînée nous-mêmes.

2 Les trois démos

Le *context encoder* cache le centre de l'image et doit le reconstruire. Dans la démo, seule la perte de reconstruction $\mathcal{L}_{rec} = \|\hat{M} \odot (x - F((1-\hat{M}) \odot x))\|_2^2$ est utilisée, la partie adversariale de l'article n'est pas reprise. La représentation évaluée est le vecteur au milieu de l'auto-encodeur (4000 dimensions). En regardant les reconstructions, on voit qu'elles sont floues : le modèle prolonge les couleurs et les contours du bord, mais ne reconstruit pas vraiment l'objet caché.

Pour la rotation, chaque image est tournée de 0, 90, 180 ou 270 degrés et le réseau (un *Network in Network*) doit retrouver l'angle. L'idée est que pour savoir si une image est à l'envers, il faut savoir où sont le ciel, le sol ou la tête d'un animal, donc comprendre un minimum ce que contient l'image. La représentation utilisée est la sortie de la couche `conv2`, de taille $192 \times 8 \times 8 = 12\,288$.

SimCLR fabrique deux versions augmentées de la même image, les fait passer dans un ResNet-50 puis dans une petite tête de projection, et utilise une perte contrastive (NT-Xent). Les deux versions d'une même image doivent avoir des représentations proches, et celles des autres images du lot servent d'exemples négatifs. On évalue la sortie du ResNet avant la projection (2048 dimensions).

Les plus proches voisins dans l'espace latent donnent déjà une idée de la qualité de chaque représentation. Avec le *context encoder*, les voisins ont surtout la même couleur et le même fond que l'image de départ : une grenouille sur fond clair donne des chiens et des oiseaux sur fond beige, et un cerf dans l'herbe donne d'autres animaux sur fond vert, mais aussi des camions. Avec la rotation, c'est plus variable : un oiseau ne donne que des oiseaux, mais un camion rouge donne des camions et des voitures mêlés à des grenouilles, et un camion de pompiers donne surtout des bateaux et des avions. La forme générale et la scène comptent donc encore beaucoup. Seul SimCLR regroupe vraiment par classe : un cargo ne donne que des navires, une voiture verte que des voitures et une grenouille que des grenouilles.

3 Protocole de comparaison

Pour comparer les méthodes, nous gardons l'encodeur figé et nous n'entraînons qu'une couche linéaire au-dessus, avec le même classifieur que dans chaque démo. Dans une première version, nous avions réutilisé sur CIFAR-10 le classifieur fourni avec le checkpoint, mais il a été appris sur les 50 000 images, ce qui avantageait CIFAR-10 par rapport à l'autre jeu de données. Dans la version finale, nous réentraînons donc tous les classifieurs de la même façon : nous calculons une seule fois les représentations (10 000 images au maximum), puis nous entraînons la couche linéaire pendant 30 époques avec Adam ($\text{lr} = 10^{-3}$, lots de 256). Nous donnons ensuite la précision top-1 et top-5 sur tout le jeu de test. Les scores sur CIFAR-10 sont donc un peu plus bas qu'avec le checkpoint (89,9 % au lieu de 92,8 % pour SimCLR).

Comme second jeu de données, nous avons pris STL-10. Il est de taille comparable à CIFAR-10, il a aussi 10 classes dont 9 en commun avec lui (le singe remplace la grenouille), et il se télécharge facilement depuis Hugging Face. Ses images font 96×96, nous les avons donc réduites à 32×32 pour qu'elles aient la même taille que les images vues par les encodeurs. Il faut noter que STL-10 n'a que 5 000 images annotées pour l'entraînement, c'est donc avec ces 5 000 images que le classifieur est appris. Les encodeurs, eux, ne sont pas réentraînés : on regarde si une représentation apprise sur CIFAR-10 reste utile sur d'autres images.

4 Prédiction de la position relative

Nous avons repris le principe de Doersch *et al.* [4], mais il a fallu l'adapter car les images de CIFAR-10 ne font que 32×32. Nous plaçons une grille de 3×3 cellules de 10 pixels à un endroit aléatoire de l'image, et dans chaque cellule nous découpons un patch de 8×8. Il reste donc environ 2 pixels d'écart entre deux patchs voisins, et nous ajoutons en plus un petit décalage aléatoire de ± 1 pixel. Le patch central est ensuite associé à chacun de ses 8 voisins, et le réseau doit dire dans laquelle des 8 positions se trouve le voisin. Une image donne donc 8 exemples, sans aucune étiquette.

L'écart et le décalage servent à éviter que le modèle triche en comparant simplement les bords des patchs ou en suivant une ligne droite d'un patch à l'autre. Pour la même raison, chaque patch est passé en niveaux de gris avec une probabilité de 0,66 : la démo 5 montre l'aberration chromatique (des franges colorées sur les bords des objets), qui varie selon la position dans l'image et pourrait donc indiquer où se trouve un patch sans regarder son contenu. Les deux patchs passent dans le même encodeur, puis leurs caractéristiques sont mises bout à bout et envoyées à une petite tête (une couche cachée de 512 neurones, 8 sorties).

Pour pouvoir comparer honnêtement avec la rotation, nous avons gardé le même encodeur (*Network in Network*), la même couche `conv2`, le même prétraitement et le même classifieur linéaire. Pour l'entraînement, nous avons repris les réglages du script de la librairie : SGD avec Nesterov (moment 0,9), $\mathrm{lr} = 0{,}1$ avec décroissance cosinus, *weight decay* de $5 \cdot 10^{-4}$, lots de 128 et 200 époques. Nous avons rencontré un problème à l'évaluation : pendant l'entraînement, l'encodeur ne voit que des patchs de 8×8, mais à l'évaluation on lui donne l'image entière, comme pour la rotation. Les statistiques de BatchNorm, calculées sur des patchs, risquaient donc de ne pas correspondre aux images entières. Pour corriger ça, nous recalculons ces statistiques sur 100 lots d'images entières à la fin de l'entraînement, sans modifier les poids.

La tâche est assez difficile pour un réseau de cette taille. Au hasard, on aurait 12,5 % de bonnes réponses. La précision sur la tâche prétexte passe de 38,5 % à la première époque à 52,6 % à la 50e, puis 54,5 % à la 100e. Elle remonte ensuite plus vite quand le taux d'apprentissage devient faible, et finit à 60,4 % après 200 époques, avec une perte qui descend de 1,53 à 1,03. Nous pensons qu'une bonne partie des erreurs vient des patchs de fond uni, comme le ciel ou la mer, pour lesquels il est impossible de deviner la position.

5 Résultats

On retrouve le même ordre sur les deux jeux de données : SimCLR est le meilleur, suivi de la rotation puis de la position relative, et le *context encoder* est loin derrière. Pour faire baisser la perte de reconstruction, il suffit de produire une image floue avec les bonnes couleurs, donc le réseau n'a pas besoin de comprendre ce qu'il y a dans l'image. Avec la rotation ou SimCLR, au contraire, il faut prendre une décision qui dépend du contenu.

Sur STL-10, toutes les méthodes perdent en précision : 9,4 points pour SimCLR, 3,3 pour la rotation, 2,7 pour la position relative et 2,0 pour le *context encoder*. Plusieurs raisons peuvent l'expliquer : les classes ne sont pas exactement les mêmes, la réduction de 96 à 32 pixels fait perdre des détails, et le classifieur n'a que 5 000 images pour apprendre au lieu de 10 000. Ce dernier point se voit bien avec la rotation et la position relative, qui atteignent toutes les deux 100 % en entraînement sur STL-10 : avec 12 288 dimensions et si peu d'images, la couche linéaire sur-apprend.

TABLE 1 – Précision en test de la classification linéaire sur la représentation figée (%). Tous les encodeurs sont pré-entraînés sur CIFAR-10 sans étiquettes, et tous les classifieurs sont réentraînés avec le même protocole.

	CIFAR-10		STL-10	
Tâche prétexte	top-1	top-5	top-1	top-5
Context encoder	48,4	92,0	46,4	91,4
Rotation	76,8	98,3	73,5	98,1
SimCLR	**89,9**	**99,7**	**80,5**	**99,0**
Position relative	72,2	97,5	69,5	97,8

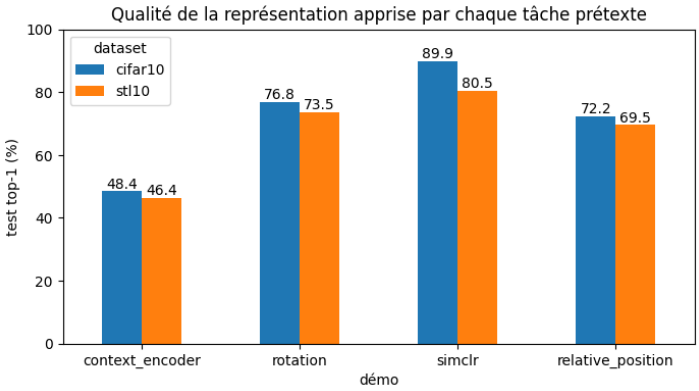


FIGURE 1 – Précision top-1 en test pour chaque tâche prétexte et chaque jeu de données.

La position relative obtient 72,2 % sur CIFAR-10, soit 4,6 points de moins que la rotation, et l'écart passe à 4,0 points sur STL-10. Comme les deux méthodes utilisent le même encodeur, la même couche et le même classifieur, cette différence vient seulement de la tâche prétexte. Nous y voyons deux raisons. D'abord, l'encodeur n'a jamais vu d'image entière pendant l'entraînement, seulement des patchs de 8×8, alors qu'on l'évalue sur des images de 32×32 ; le recalcul des statistiques de BatchNorm ajuste les moyennes, mais ne change pas ce que les filtres ont appris sur des patchs. Ensuite, sur des patchs aussi petits, beaucoup de paires ne contiennent pas assez d'information pour savoir où est le voisin, ce que montre la précision de 60,4 % sur la tâche prétexte. La position relative reste quand même largement au-dessus du *context encoder* (+23,8 points sur CIFAR-10), ce qui montre qu'elle apprend des caractéristiques utiles.

Il faut enfin nuancer l'avance de SimCLR : il utilise un ResNet-50, beaucoup plus gros que le *Network in Network* de la rotation et de la position relative, et une partie de son avance vient de là.

6 Conclusion

Ce TP nous a montré que le choix de la tâche prétexte change beaucoup la qualité de la représentation. Les tâches qui obligent le modèle à comprendre le contenu de l'image, comme la rotation ou l'apprentissage contrastif, donnent des représentations qu'un simple classifieur linéaire peut exploiter, alors que la reconstruction de pixels donne de moins bons résultats. En implémentant la position relative, nous avons surtout compris que les détails de la tâche sont importants : sans écart entre les patchs, sans décalage et sans passage en gris, le réseau peut trouver des raccourcis de bas niveau au lieu d'apprendre quelque chose d'utile. Avec ces précautions, notre implémentation arrive à 72,2 % sur CIFAR-10 et 69,5 % sur STL-10, derrière la rotation mais loin devant le *context encoder*. Il serait intéressant d'essayer des patchs plus grands sur des images de meilleure résolution, comme dans l'article original, pour voir si l'écart avec la rotation se réduit.

Annexes

A. Code source

Notebook complet (démos, script de comparaison et tâche de position relative) : `https://github.com/Harena21/tp-ssl`

B. Sources

Références

[1] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, A. Efros. *Context encoders : Feature learning by inpainting*. CVPR, 2016.

[2] S. Gidaris, P. Singh, N. Komodakis. *Unsupervised representation learning by predicting image rotations*. ICLR, 2018.

[3] T. Chen, S. Kornblith, M. Norouzi, G. Hinton. *A simple framework for contrastive learning of visual representations*. ICML, 2020.

[4] C. Doersch, A. Gupta, A. Efros. *Unsupervised visual representation learning by context prediction*. ICCV, 2015.

[5] A. Coates, A. Ng, H. Lee. *An analysis of single-layer networks in unsupervised feature learning*. AISTATS, 2011.

[6] Cours CS294-158 Deep Unsupervised Learning, UC Berkeley, 2020, et dépôt `https://github.com/ckurtz/cs294-158-ssl`.